

CHAP 11(2): numpy배열 데이터 처리 분석2

- 빅데이터 란?
- 빅데이터와 데이터 분석
(데이터의 가치, 블랙홀 관측과 데이터 분석)
- 난수 생성하기, 균일 분포 난수 생성하기,
정규 분포 난수 생성,
- 평균값과 중앙값 계산하기

[참고도서]

- 따라하며 배우는 파이썬과 데이터과학
- 옴 데이터분석과 머신러닝

빅데이터란?

- 빅데이터(**big data**)는 기존 데이터베이스 관리 도구의 능력을 넘어서는 대량의 정형, 비정형 데이터 집합으로부터 가치를 추출하고 결과를 분석하는 기술을 말한다.
- 빅데이터는 흔히 양**volume**과 속도**velocity**, 그리고 종류**variety**의 첫 글자를 따서 **3V**로 그 특징을 표시.
 - 양(**Volume**) : 테라바이트에서 페타바이트, 엑사바이트 이상의 데이터를 저장하여 분석하고 가공. 페타바이트 데이터는 **1015** 바이트 크기의 데이터인데, 이 이상의 데이터는 엄청난 물리적 데이터 크기로 인하여 이를 분석 및 처리하는 데 어려움이 따른다.
 - 속도(**Velocity**) : 사물인터넷을 비롯한 센서와 다양한 모바일 기기, 소셜미디어 등으로 부터 발생하는 매우 빠른 속도의 실시간적인 데이터 종류의 증가를 그 특징으로 볼 수 있다.
 - 종류(**Variety**) : 빅데이터는 로그기록, 소셜미디어, 실시간 시청 기록 등의 다양한 기기 와 매체를 통해서 데이터가 축적되므로 텍스트 정보 이외에도 멀티미디어 정보와 같은 비정형화된 데이터가 많은 부분을 차지한다.

빅데이터와 데이터 분석

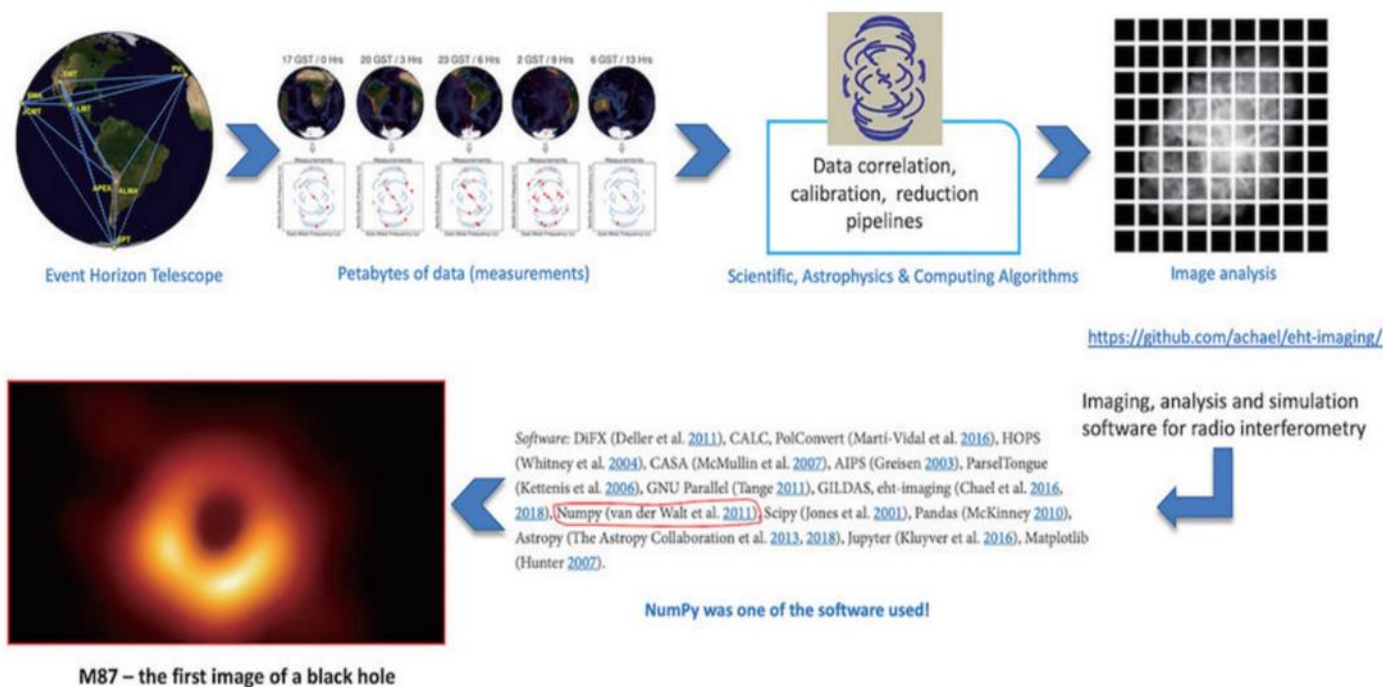
- 과거에도 데이터의 중요성은 알려져 있었으나 데이터를 획득하여도 오늘날처럼 저장하고 가공하여 활용하기 위한 기술이 확보되지 못함.
- 반면, 최근에는 다양한 측정 장치(센서)로부터 실시간으로 데이터가 수집되며, 이 방대한 양의 데이터를 저장하고 활용 가능한 리소스가 됨.
- 빅데이터 수집과 활용 사례.
 - 제주관광공사 연구센터 : 관광 빅데이터를 구축하고 시스템화 → 관광객들의 이동 패턴과 결제 장소에 따른 주요 상권 분석 등 관광상품 개발에 필요한 여러 빅데이터 확보
 - 아마존닷컴 : 모든 고객의 구매내역을 데이터베이스에 기록 → 이 기록을 분석해 소비자의 소비취향과 관심사를 파악 → 고객별로 '추천 상품' 표시
 - 독일 국가대표팀 : **SAP**와 협업하여 훈련과 실전 경기에 '**SAP** 매치 인사이트'를 도입 → 선수들에게 부착된 센서를 통해 운동량, 순간속도, 심박수, 슈팅 동작 등 방대한 비정형 데이터를 수집 → 분석한 결과를 감독과 코치의 태블릿으로 전송 → 데이터를 기반으로 전술을 짜도록 도와줌

빅데이터와 데이터 분석

- 데이터의 가치
 - 최근에는 기업과 정부 기관 등에서 이와 같은 적극적인 데이터 수집과 활용이 이루어지고 있는 추세이다.
 - 하지만 아직도 많은 데이터는 가공되지 않은 원시 데이터 *raw data* 형태로 기업과 기관의 저장장치에서 잠들어 있다.
 - 원시 데이터를 의미 있는 데이터로 가공하기 위해서는 데이터를 처리할 수 있는 시스템과 데이터에 대한 깊은 통찰력을 가진 데이터 과학 전문가가 필수적이다.

빅데이터와 데이터 분석

- 블랙홀 관측과 데이터 분석
 - 수집된 데이터는 촬영 시간과 좌표 등의 여러 가지 정보를 포함하고 있는 디지털 데이터이다. 이 데이터의 총 분량은 5페타바이트로 일반적인 초고속 인터넷망으로 전송이 불가능한 대용량의 데이터였다.



The role of NumPy in Black Hole imaging

(출처: 으뜸 데이터 분석과 머신러닝)

빅데이터와 데이터 분석

- 따라서 연구팀은 전 세계의 데이터를 비행기를 이용하여 한 곳으로 수집한 후 데이터의 상관도를 분석하고, 측정값 보정 작업과 여러 전처리 작업을 거친 다음 이미지 처리 작업을 수행하였다.
- 이 과정에서 대용량의 데이터 처리 작업이 수행되었으며, 파이썬의 넘파이 패키지가 중요한 역할을 담당하였다.
- 21세기 과학의 업적 중 하나인 블랙홀의 그림자 이미지 생성 작업은 데이터 분석과 처리 작업이 천체 물리학과 협업을 통해서 얻은 업적이라고 볼 수 있다.

빅데이터와 데이터 분석

- 데이터 양이 많을수록,
넘피 배열 처리시간이 리스트보다 훨씬 빠르다.
 - 넘피 배열 처리시간. (벡터화 연산)

```
import numpy as np
import time
start= time.time()
data=np.random.randint(1,100, size=(10000000))
data=data+10
end= time.time()
print('소요시간', end-start)
```

😊출력 화면

소요시간 0.08274316787719727

넘피배열 처리시간 vs. 리스트 처리시간

빅데이터와 데이터 분석

- 리스트 처리시간 (원소별로 따로 접근)

```
import random
import time
start= time.time()
data=[ ]
for n in range(10000000):
    data.append( random.randint(1,100))

for i in range(len(data)) :
    data[i]= data[i]+10

end= time.time()
print('소요시간', end-start)
```

소요시간

0.8247931003570557

😊출력 화면

소요시간 6.88657283782959

난수 생성하기

- 실제 관측 데이터는 장시간의 관측이 필요하므로, 실세계 데이터와 유사한 특성을 가지는 가상의 데이터를 만들어 분석하고 시뮬레이션한 후에 실제 데이터 분석에 사용한다.
- 난수 생성으로 실험 데이터를 만든다.
- 넘파이 난수 생성은 **random** 서브 모듈에서 제공하는 함수들을 사용

함수 이름	설명
rand()	균등 분포의 난수 표본을 추출
randn()	표준 정규 분포의 난수 표본을 추출
randint()	최소값과 최대값을 인자로 받아, 주어진 범위 내의 정수 난수를 생성.
normal()	평균과 표준편차를 인수로 받아, 정규분포의 난수 표본을 추출
seed()	난수의 시드(seed)를 설정. 실행 때마다 동일한 난수 값이 생성됨.

난수 생성하기

- 난수의 시드(seed)를 설정하면, 늘 같은 난수가 생성된다.

```
import numpy as np  
np.random.seed(n)
```

#인수 n은 0이상의 정수. 난수의 seed 설정

- 예) 시드 설정해서 난수 생성하기
 - np.random.seed(42) #시드 설정
시드를 설정하면, 실행 때마다 매번 동일한 난수를 얻을 수 있음.
 - np.random.rand(5) #인수는 난수 생성 개수.
0.0~1.0 사이의 난수 5개 생성. 균등분포 난수

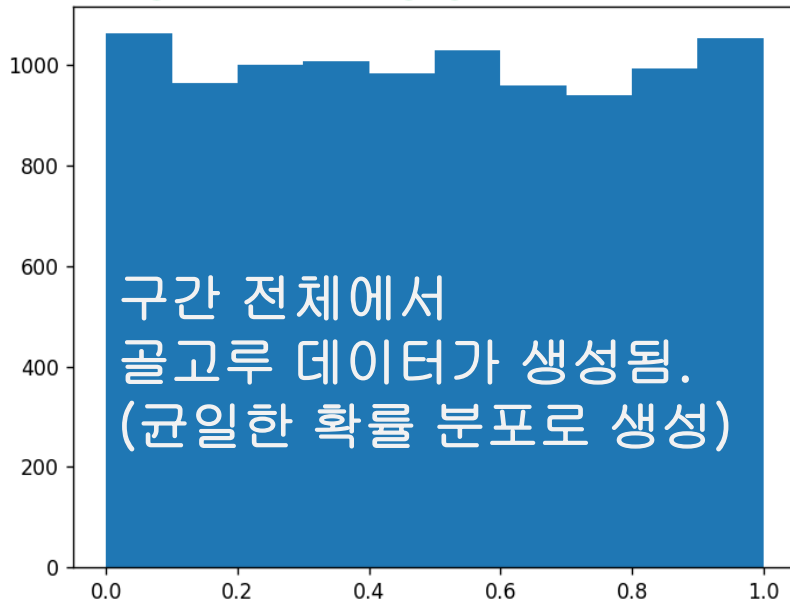
```
>>> np.random.seed(42)  
>>> np.random.rand(5)  
array([0.37454012, 0.95071431, 0.73199394, 0.59865848,  
0.15601864])  
>>>
```

난수 생성하기

- 균등 분포 vs. 정규 분포

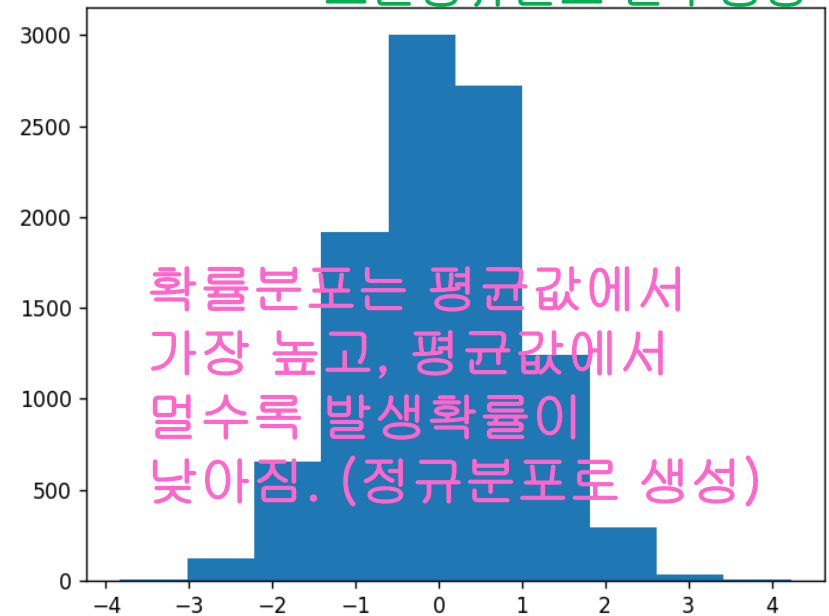
```
>>> data=np.random.rand(10000)
>>> plt.hist(data, bins=10)
>>> plt.show()
```

0~1 사이의
균등분포 난수생성



```
>>> data=np.random.randn(10000)
>>> plt.hist(data, bins=10)
>>> plt.show()
```

평균 0, 표준편차 1
표준정규분포 난수생성



균일 분포 난수 생성하기

- `np.random.rand(3,5)` **#인수는 행, 열의 크기를 나타냄**
0.0~1.0 사이의 난수로 2차원 배열(3행5열) 생성

```
>>> np.random.rand(3,5)
array([[0.37454012, 0.95071431, 0.73199394, 0.59865848, 0.15601864],
       [0.15599452, 0.05808361, 0.86617615, 0.60111501, 0.70807258],
       [0.02058449, 0.96990985, 0.83244264, 0.21233911, 0.18182497]])
```

- 10~20까지 사이의 난수 5개 생성하기. 1차원 배열 생성.

```
>>> a=10
>>> b=20
>>> (b-a)* np.random.rand(5)+a
array([16.79568294 18.65076548 18.9333994 14.2217218 18.59522647])
>>>
>>> ( (b-a)* np.random.rand(5)+a ).round(2) #소수점이하 2자리로 정리
array([17.35 11.48 11.06 10.67 10.29])
```

균일 분포 난수 생성하기

- `np.random.randint(1,7, size=10)` #1~6 사이의 정수 난수 생성.
(1차원 배열 생성. 원소 10개)

```
>>> np.random.randint(1,7, size=10)
array([3, 5, 1, 2, 4, 2, 6, 1, 4, 1])
```

- `np.random.randint(1,7, size=(3,10))` #2차원 정수 배열(3행10열) 생성

```
>>> np.random.randint(1,7, size=(3,10))
array([[6, 5, 5, 6, 3, 5, 5, 2, 2, 6],
       [2, 6, 6, 6, 2, 6, 6, 2, 2, 4],
       [1, 5, 2, 2, 2, 4, 1, 4, 6, 2]])
```

정규 분포 난수 생성

- `randn()`는 평균은 0, 표준편차 1이 되는 표준정규분포 난수를 생성.

```
>>> np.random.randn(4)
array([ 1.12060892, -0.07748086,  0.30573564, -0.00143797])
```

- 이 함수를 사용하여 평균값 m , 표준편차 s 인 정규분포 난수를 만들기.

```
>>> m=10 #평균값
>>> s=2  #표준편차
>>> m + s * np.random.randn(3,4)
array([[ 9.7880129 ,  7.01891658, 10.27340637, 10.43525388],
       [ 8.55337197, 11.96153022,  8.98679539,  6.63378293],
       [ 7.08173711, 13.75126338,  9.29930925, 10.83303722]])
```

평균과 중앙값 계산하기

- 예1) 대학생 10,000명의 키와 몸무게, 나이를 저장한다. (균등분포 난수)
 - 2차원 배열 (10,000행 3열)
 - 키 155~190, 몸무게 55~120, 나이 19~30인 난수 생성으로 데이터 저장. `random.randint()` 사용
 - 생성된 데이터의 평균과 중앙값을 구해보자.

😊출력 화면

신장 평균값: 171.9026
신장 중앙값: 172.0
신장 표준편차: 10.14396930397564

체중 평균값: 87.0346
체중 중앙값: 87.0
체중 표준편차: 18.6554550424266

나이 평균값: 24.0176
나이 중앙값: 24.0
나이 표준편차: 3.1710708349073498

평균과 중앙값 계산하기

샘플 소스

```
import numpy as np
```

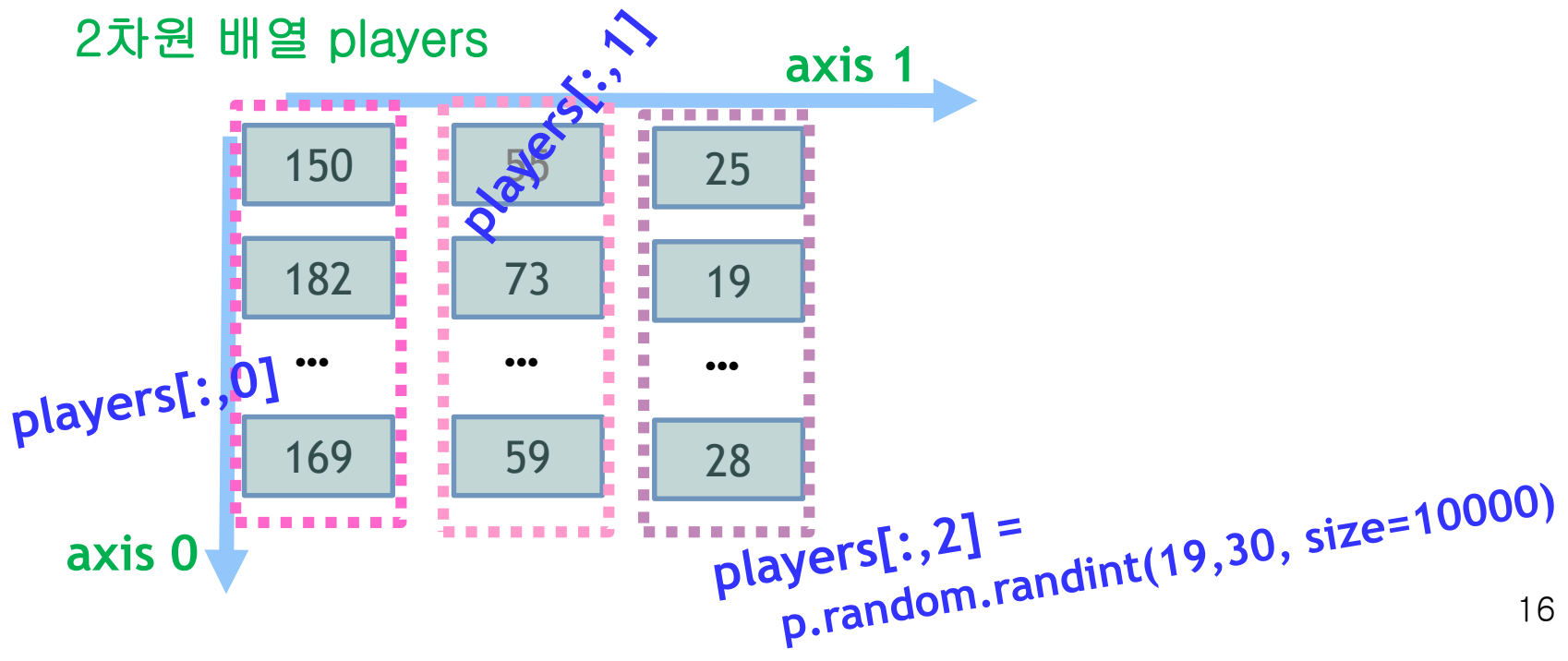
```
players = np.zeros( (10000, 3) )
```

```
players[:, 0] = np.random.randint(155,190, size=10000)
```

```
players[:, 1] = np.random.randint(55,120, size=10000)
```

```
players[:, 2] = np.random.randint(19,30, size=10000)
```

2차원 배열 players



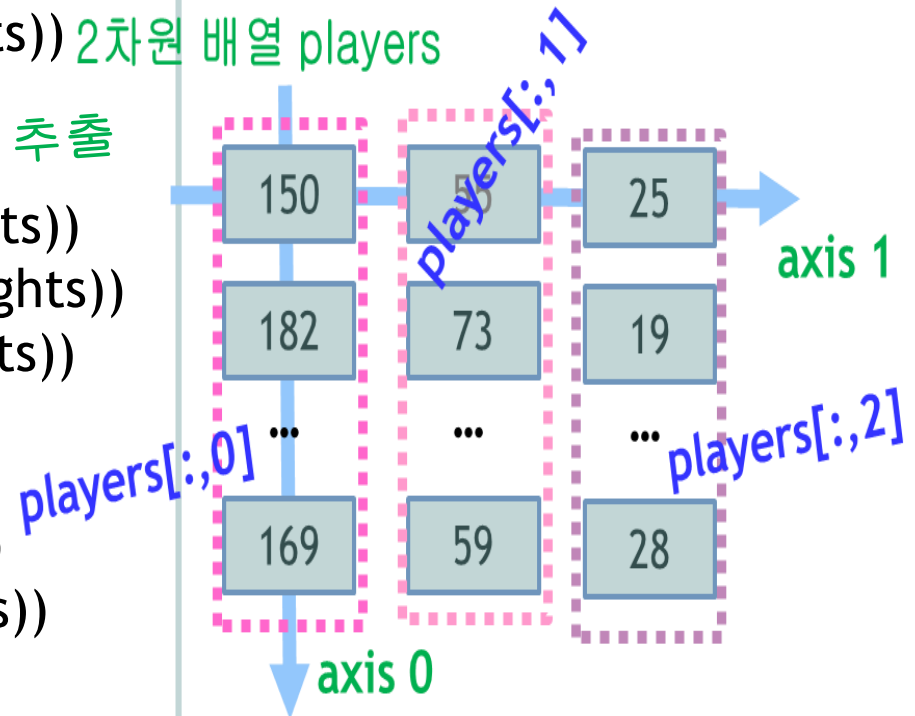
평균과 중앙값 계산하기

#열 별로 추출(슬라이싱) 하여, 따로 계산

```
heights = players[:, 0] #첫번째 열을 추출 (키 값 추출)  
print('신장 평균값:', np.mean(heights)) #이것에 대해 계산  
print('신장 중앙값:', np.median(heights))  
print('신장 표준편차:', np.std(heights))
```

```
weights = players[:, 1] #두번째 열을 추출  
print('체중 평균값:', np.mean(weights))  
print('체중 중앙값:', np.median(weights))  
print('체중 표준편차:', np.std(weights))
```

```
ages = players[:, 2]  
print('나이 평균값:', np.mean(ages))  
print('나이 중앙값:', np.median(ages))  
print('나이 표준편차:', np.std(ages))
```



평균과 중앙값 계산하기

#axis=0지정으로, 배열의 같은 열 끼리 계산.

```
평균= players.mean(axis=0) #np.mean(players, axis=0) 동일  
중앙값= np.median(players, axis=0) #축 0을 지정해서,  
표준편차= players.std(axis=0) #열끼리 계산
```

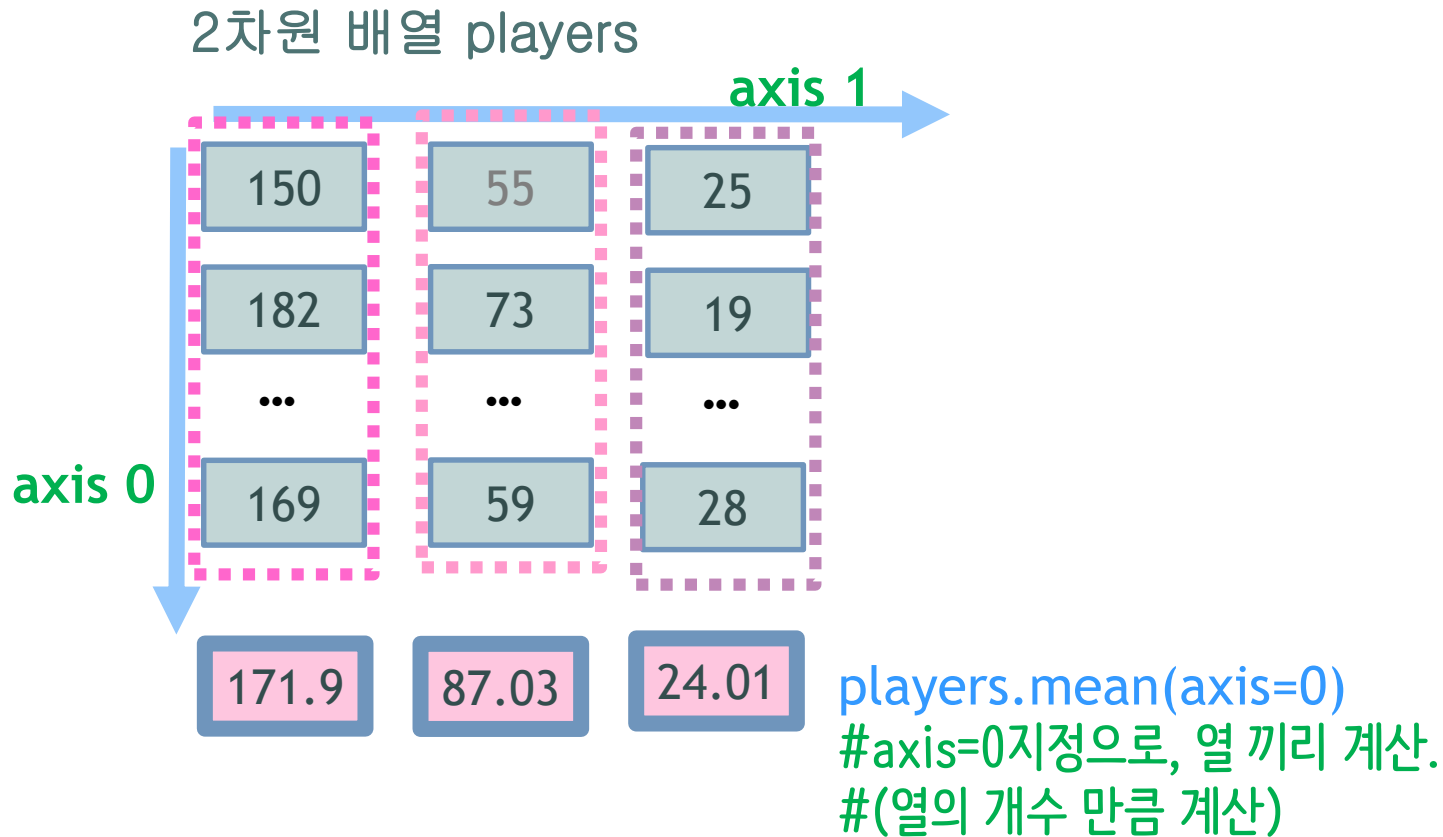
```
print('신장 평균값:', 평균[0] )  
print('신장 중앙값:', 중앙값[0] )  
print('신장 표준편차:', 표준편차[0])
```

```
print('체중 평균값:', 평균[1])  
print('체중 중앙값:', 중앙값[1] )  
print('체중 표준편차:', 표준편차[1])
```

```
print('나이 평균값:', 평균[2] )  
print('나이 중앙값:', 중앙값[2])  
print('나이 표준편차:', 표준편차[2])
```



평균과 중앙값 계산하기



```
>>> players.mean(axis=0) #np.mean(players, axis=0) 동일  
array([174.89178121, 70.15797325, 22.02315149])  
>>>
```

평균과 중앙값 계산하기

- 예2) 대학생 10,000명의 키와 몸무게, 나이를 저장한다. (균등분포 난수)
 - 2차원 배열 (10,000행 3열)
 - 키 155~190, 몸무게 55~120, 나이 19~30인 난수 생성으로 데이터 저장. `random.rand()` 사용
 - 생성된 데이터의 평균과 중앙값을 구해보자.

😊출력 화면

신장 평균값: 172.5702877163928
신장 중앙값: 172.7163789478241
신장 표준편차: 10.121442815010397

체중 평균값: 87.75819259426979
체중 중앙값: 87.95940364343166
체중 표준편차: 18.670305667724236

나이 평균값: 24.0301
나이 중앙값: 24.0
나이 표준편차: 3.185654405298855

평균과 중앙값 계산하기

```
import numpy as np

players = np.zeros( (10000, 3) )

a= 155; b=190
players[:, 0] = (b-a) * np.random.rand(10000) + a

a= 55; b=120
players[:, 1] = (b-a) * np.random.rand(10000) + a

a=19; b=30
players[:, 2] = np.floor((b-a) * np.random.rand(10000)) + a
#실수의 바로 아래 정수로 내림
```

평균과 중앙값 계산하기

```
import numpy as np

players = np.zeros( (10000, 3) )
players[:, 0] = 10 * np.random.randn(10000) + 175
players[:, 1] = 10 * np.random.randn(10000) + 70
players[:, 2] = np.floor(10 * np.random.randn(10000)) + 22
                #실수의 바로 아래 정수로 내림

heights = players[:, 0]
print('신장 평균값:', np.mean(heights))
print('신장 중앙값:', np.median(heights))

weights = players[:, 1]
print('체중 평균값:', np.mean(weights))
print('체중 중앙값:', np.median(weights))

ages = players[:, 2]
print('나이 평균값:', np.mean(ages))
print('나이 중앙값:', np.median(ages))
```

평균과 중앙값 계산하기

- 예3) 대학생 10,000명의 키와 몸무게, 나이를 배열에 저장한다.
 - 2차원 배열 (10,000행 3열)
 - 키 175, 몸무게 70, 나이 22가 평균인 난수 생성으로 데이터를 저장.
 - 각각 표준편차는 10
 - `random.randn()` 사용
 - 생성된 데이터의 평균과 중앙값을 구해보자.

😊출력 화면

```
신장 평균값: 175.18541950036365  
신장 중앙값: 175.13255704824087  
체중 평균값: 69.98253499420532  
체중 중앙값: 69.92951796146212  
나이 평균값: 21.5943  
나이 중앙값: 22.0
```

평균과 중앙값 계산하기

```
import numpy as np

players = np.zeros( (10000, 3) )
players[:, 0] = 10 * np.random.randn(10000) + 175
players[:, 1] = 10 * np.random.randn(10000) + 70
players[:, 2] = np.floor(10 * np.random.randn(10000)) + 22
                #실수의 바로 아래 정수로 내림

heights = players[:, 0]
print('신장 평균값:', np.mean(heights))
print('신장 중앙값:', np.median(heights))

weights = players[:, 1]
print('체중 평균값:', np.mean(weights))
print('체중 중앙값:', np.median(weights))

ages = players[:, 2]
print('나이 평균값:', np.mean(ages))
print('나이 중앙값:', np.median(ages))
```

평균과 중앙값 계산하기

- 예4) 대학생 10,000명의 키와 몸무게, 나이를 저장한다.
 - 2차원 배열 (10,000행 3열)
 - 키 175, 몸무게 70, 나이 22가 평균인 난수 생성으로 데이터 생성.
 - 표준편차는 모두 10
 - `random.normal()` 사용
 - 생성된 데이터의 평균과 중앙값을 구해보자.

😊출력 화면

```
신장 평균값: 174.9384149122437
신장 중앙값: 174.9061051211413
체중 평균값: 69.97035286223269
체중 중앙값: 70.03055618269437
나이 평균값: 22.043231710341004
나이 중앙값: 22.027693570132065
```

평균과 중앙값 계산하기

```
import numpy as np
```

```
players = np.zeros( (10000, 3) )
```

```
players[:, 0] = np.random.normal(loc=175,scale=10,size=10000)
```

```
players[:, 1] = np.random.normal(loc=70,scale=10,size=10000)
```

```
players[:, 2] = np.random.normal(loc=22,scale=10,size=10000)
```

```
heights = players[:, 0]
```

```
print('신장 평균값:', np.mean(heights))
```

```
print('신장 중앙값:', np.median(heights))
```

```
weights = players[:, 1]
```

```
print('체중 평균값:', np.mean(weights))
```

```
print('체중 중앙값:', np.median(weights))
```

```
ages = players[:, 2]
```

```
print('나이 평균값:', np.mean(ages))
```

```
print('나이 중앙값:', np.median(ages))
```